Practice work #14

Student name: Abdykamat Adilet

Specialty: IS

Instructor’s full name: Aasso Ziro

Year of completion: 2026

Almaty 2026

1) Objective of the practice work:

The objective of this practice work is to understand and apply dynamic memory allocation in C programming. The main goal is to learn how to allocate, manage, resize, and free memory during program execution using standard library functions such as malloc(), calloc(), realloc(), and free(). This work also aims to develop practical skills in avoiding common memory-related errors such as memory leaks, null pointer dereferencing, and improper memory handling.

2) Brief theoretical background:

In C programming, memory can be managed in two main ways: static allocation and dynamic allocation. Static memory allocation is done at compile time and has a fixed size, while dynamic memory allocation is performed at runtime using the heap, allowing flexible memory usage. Dynamic memory is handled using pointers and the following functions:

malloc(size) allocates a block of memory of a given size in bytes. The allocated memory is uninitialized and may contain garbage values.

calloc(num, size) allocates memory for multiple elements and initializes all bytes to zero.

realloc(ptr, new_size) resizes a previously allocated memory block. It may move the memory to a new location.

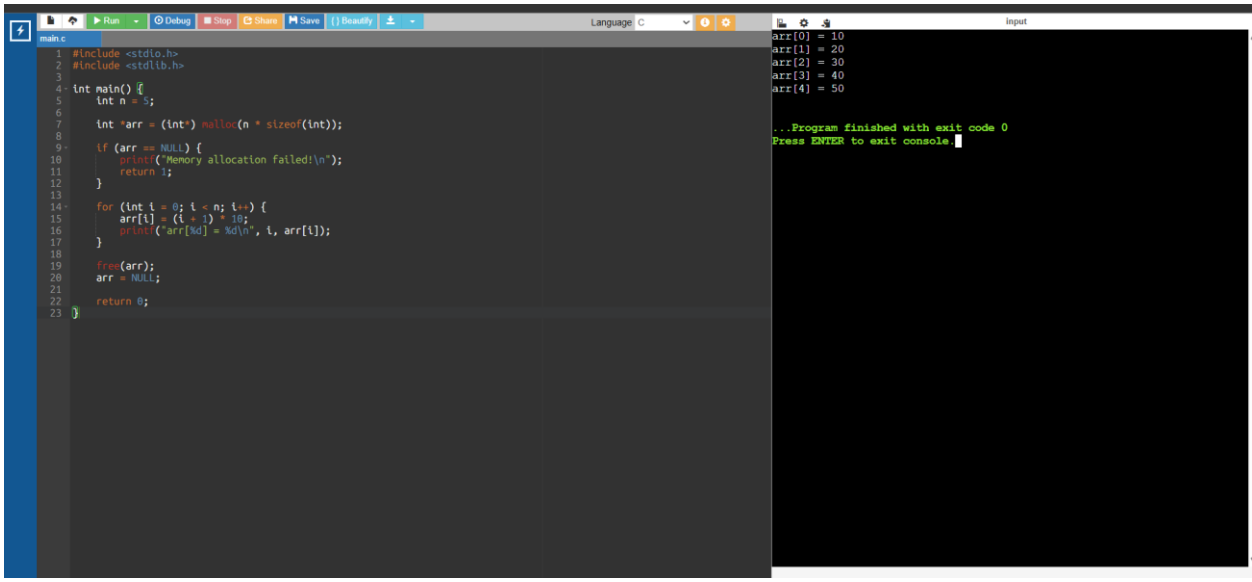
free(ptr) releases allocated memory back to the system. Proper usage of these functions requires checking whether the returned pointer is NULL, which indicates allocation failure. Additionally, after freeing memory, it is considered good practice to set the pointer to NULL to prevent dangling pointer issues.

3) Step-by-step procedure:

First, the required libraries stdio.h and stdlib.h were included in the program to enable input/output operations and memory management functions. Next, a pointer variable was declared to store the address of dynamically allocated memory. The memory allocation functions (malloc(), calloc()) were used to allocate memory for arrays. After allocation, the program checked whether the pointer was NULL to ensure that memory was successfully allocated. Then, the allocated memory was used to store and manipulate data, such as assigning values to array elements and printing them to the console. In tasks involving resizing, the realloc() function was used to change the size of the allocated memory block. A temporary pointer was used to safely handle reallocation and prevent memory loss in case of failure. Finally, all allocated memory was released using the free() function, and pointers were set to NULL to ensure safe memory handling.

4)Exercises: The exercises were performed using the OnlineGDB compiler.

1. In this exercise, memory was dynamically allocated for an array of integers using malloc(). Since malloc() does not initialize memory, the program manually assigned values to each element. The array was then printed to verify correct allocation and usage.

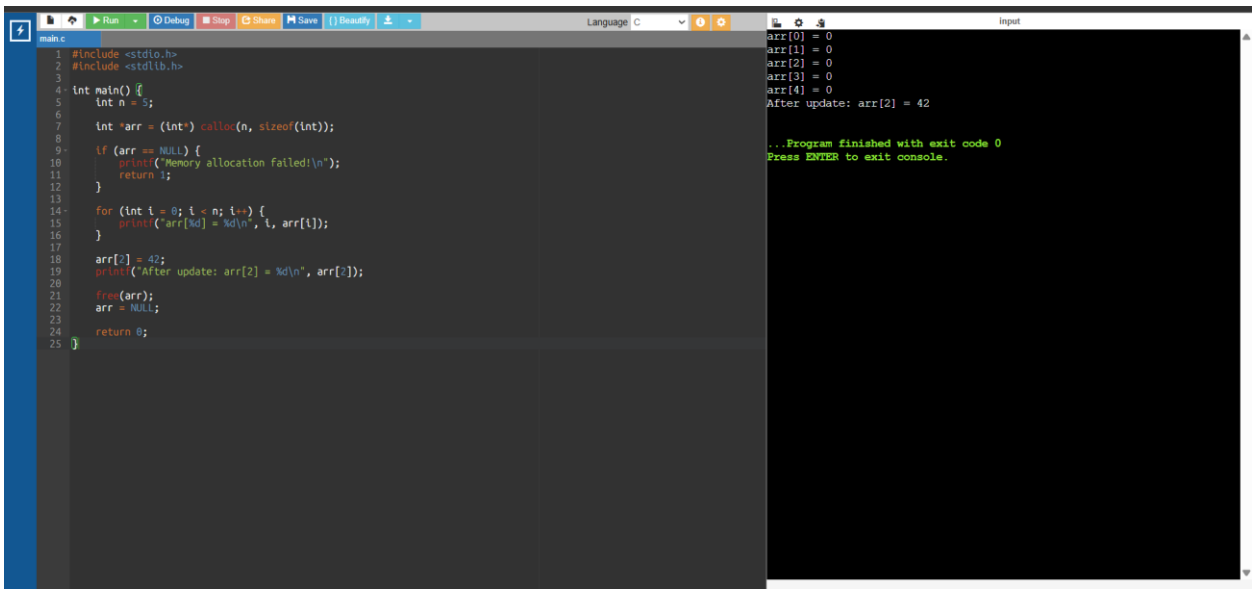


Result: The program successfully allocated memory and stored values in the array. The output displayed correctly initialized elements:

arr[0]=10
arr[1]=20
arr[2]=30
arr[3]=40
arr[4] = 50

https://onlinegdb.com/g5dr96hJb

2. In this task, calloc() was used to allocate memory for an array. Unlike malloc(), this function initializes all elements to zero. The program printed the initial values to confirm zero initialization, then modified one element.

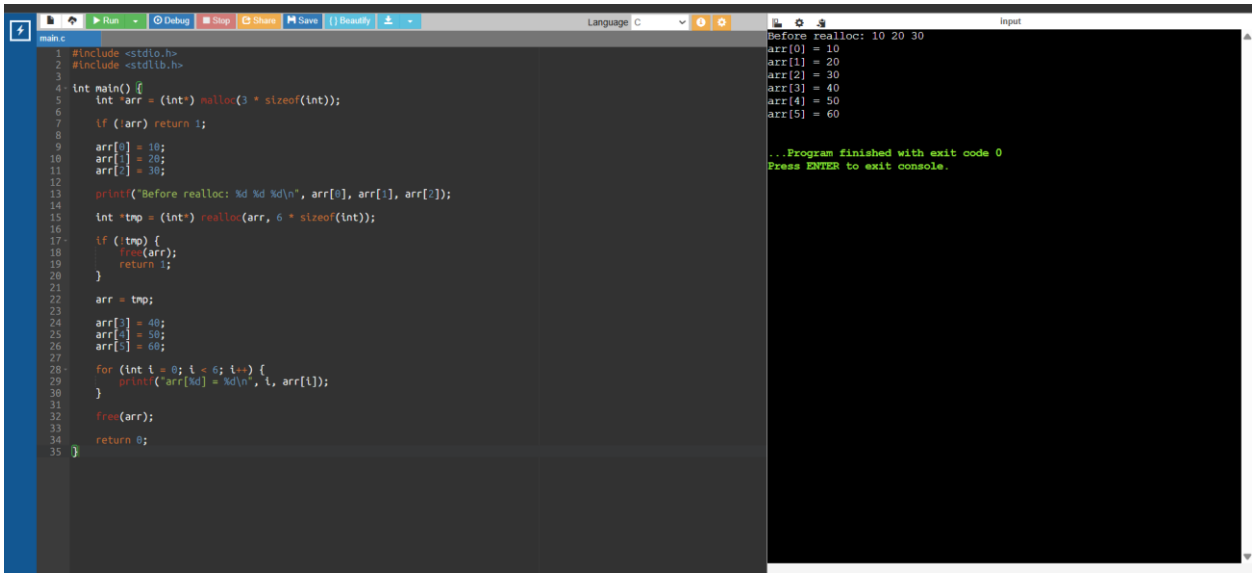


Result: The output confirmed that all elements were initially zero, and after modification, the updated value was displayed:

arr[0]=0
arr[1]=0
arr[2]=0
arr[3]=0
arr[4]=0
After update: arr[2] = 42

https://onlinegdb.com/xm5RC0AYl

3. In this exercise, memory was first allocated for 3 integers using malloc(). Then, realloc() was used to increase the size of the array to hold 6 integers. A temporary pointer was used to ensure safe reallocation.

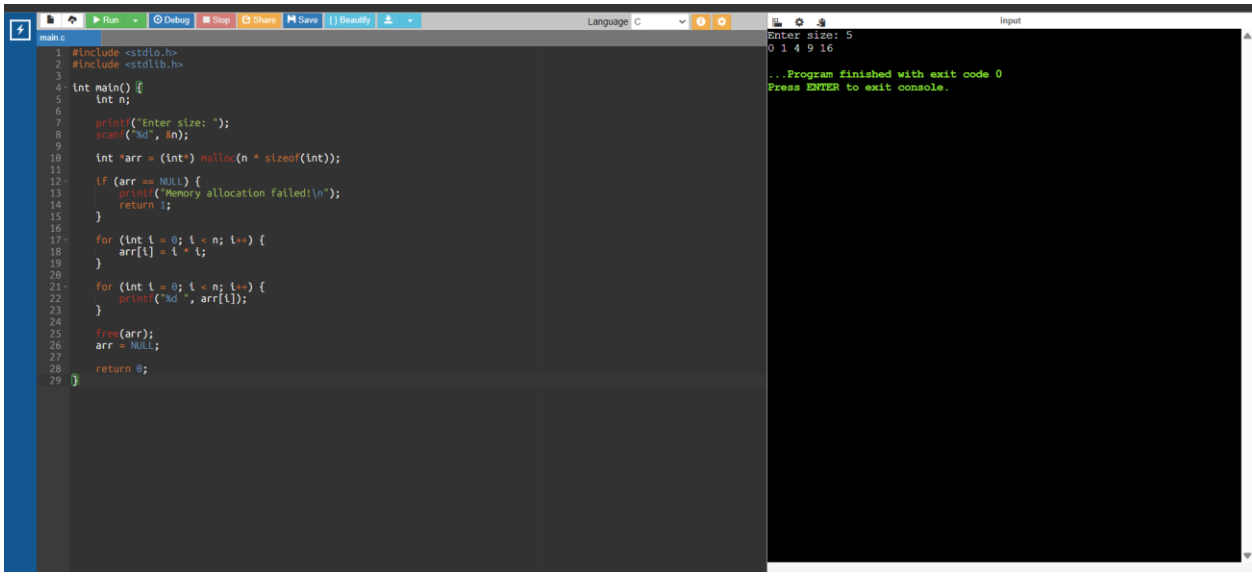


Result: The original values were preserved, and new values were successfully added:

Before_realloc:10-20-30
arr[0]=10
arr[1]=20
arr[2]=30
arr[3]=40
arr[4]=50
arr[5] = 60

https://onlinegdb.com/4plXmYLsx

4. This task required allocating memory based on user input. The program ensured safety by checking if the pointer was NULL. The array was filled with the squares of indices and printed.



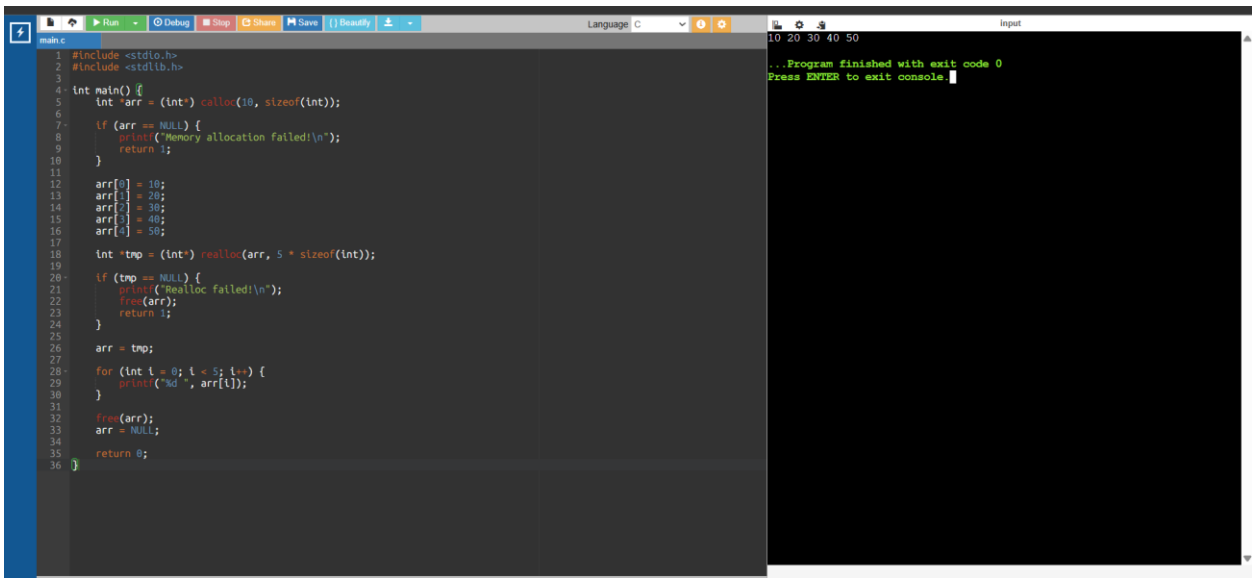
Result: For input n = 5, the output was:

0 1 4 9 16

The program handled memory safely and avoided errors.

https://onlinegdb.com/oR6KzRphYp

5. Memory was initially allocated for 10 integers using calloc(). Only the first 5 elements were assigned values. Then, realloc() was used to reduce the size of the array to 5 elements while preserving existing data.



Result: The output confirmed that only the first 5 elements remained and data was preserved:

10 20 30 40 50
https://onlinegdb.com/9v0IV4ZKj

Conclusion: In this practice, dynamic memory allocation in C was successfully explored and implemented. The functions malloc(), calloc(), and realloc() were used to manage memory efficiently, while free() ensured proper cleanup of allocated resources. The work demonstrated the importance of checking for allocation errors and following best practices to prevent memory-related issues.